

TP2 Altergrad

1.)

The square mask is used in the attention mechanism, in order to consider the words that appear before the target word in the sequence. It masks what comes after. How do we do that ? Actually we are assigning to the words that are coming next the value $-\infty$, to be sure they are not influencing the prediction.

One main detail that makes a strong difference between RNN strategies previously used and Transformer architecture's is that the Transformer parallelizes all the tokens of one sentence, and so there's no sequence notion as there is in RNN's. So the function PositionalEncoding encodes the order in the computations ! It makes context more understandable.

2.) For classification task the model outputs a probability p computing the probability to be in the class 0 or 1. But the language modeling model outputs a probability on the set of words that comes next. The tasks are different.

3.)

Language Modeling

there are three different blocs that we need to take into consideration, the Embedding layer, the Transformer and the Language Modeling Head.

For the Embedding layer, each word needs to be process into a $d_{model} = n_{hid}$ dimensional vector which gives us $n_{token}n_{hid}$ parameters.

Specific to the Transformer, we have n_{layers} Multi-Head Attention that have each three projections (query, key, and value) and one projection to combine all heads:

which give us $4n_{hid}^2n_{layers}$ parameters.

We have also a Feed-Forward Network consisting of two layers of $n_{hid}^2n_{layers}$ parameters.

Finally the Language Modeling Head consists on a final hidden states that project back the vectors to the vocabulary space which gives us $n_{token}n_{hid}$ parameters. In total we therefore have:

$$N_{paramLanguageModeling} = 2n_{token}n_{hid} + 6n_{hid}^2n_{layers}$$

Classification model: According to what we said previously, only the final head changes, we want to do the classification over $n_{classes}$ which gives us $n_{classes}n_{hid}$ parameters. In total, we will have $N_{paramClassification} = n_{token}n_{hid} + n_{classes}n_{hid} + 6n_{hid}^2n_{layers}$

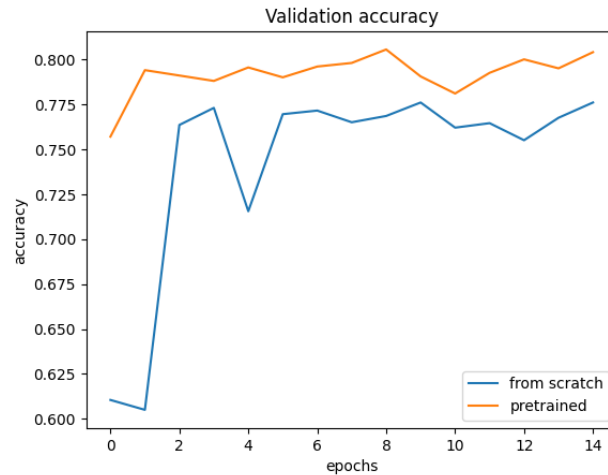


Figure 1: Accuracy evolution during training

4.) As we expected, the pre trained model allows us to achieve high accuracy much faster than the model trained from scratch. We only need one epoch to see the difference. What's more, the training phase of the pre trained model appears more stable (see curve above). Finally even at the last epochs of our training the pretrained model outperforms the model built from scratch.

5.) One strong limitation of our language model is that we have imposed unidirectional processing from left to right, which means the model proposes the next word as the most likely word to appear considering the previous words in terms of probability. But sometimes it's better to have the whole sentence in order to understand the whole meaning of something. That's why for instance BERT model which is bidirectional offers to overcome this limitation and gets a better meaning of the whole sentence. Technically speaking BERT mask a word and predicts it using it's surroundings which intuitively give us ground to assume that it captures better semantics and context.